BPFGuard: Multi-Granularity Container Runtime Mandatory Access Control

Hui Lu, Xiaojiang Du, *Fellow, IEEE*, Dawei Hu, Shen Su, and Zhihong Tian

***Abstract*—The adoption of container-based cloud computing services has been prevalent, especially with the introduction of Kubernetes, which enables the automated deployment, scaling, and administration of applications in containers, hence boosting the popularity of containers. As a result, researchers have placed greater emphasis on container runtime security, notably investigating the efficacy of traditional techniques such as Capabilities, Seccomp, and Linux security modules in guaranteeing container security. However, due to the limitations imposed by the container environment, the results have been unsatisfactory. In addition, eBPF-based solutions face the problem of being unable to quickly load policies and affect real-time operations when faced with newer kernel vulnerabilities. This paper investigates the limitations of existing container security mechanisms. Additionally, it examines the specific constraints of these mechanisms in Kubernetes environments. The paper classifies container monitoring and obligatory access control into three distinct categories: system call access control, LSM hook access control, and kernel function access control. Therefore, we propose a technique for regulating container access with a variety of granularity levels. This technique is executed using eBPF and is tightly integrated with Kubernetes to collect relevant meta-information. In addition, we suggest implementing a consolidated routing method and employing function tail call chaining to overcome the limitation of eBPF in enforcing mandatory access control for containers. Lastly, we conducted a series of experiment to verify the effectiveness of the system's security using CVE-2022-0492 and to benchmark the system that had BPFGuard enabled. The results indicate that the average performance loss increased merely by 2.16%, demonstrating that there are no adverse effects on the container services. This suggests that greater security can be achieved at a minimal cost.**

***Index Terms*—Container securiry, enforcement, Kubernetes, eBPF.**

I. Introduction

CONTAINER technology has gained popularity in cloud computing due to its efficient deployment methods and convenient management. Kubernetes has become the dominant platform for managing and orchestrating containerised applications, establishing itself as the standard in the field. Nevertheless, as containers are being widely used and their numbers are growing, security problems have become more prominent, particularly in Kubernetes clusters where container security issues are more intricate.

Initially, researchers explored the possibility of directly implementing conventional Linux security mechanisms, such as Capabilities [1], Seccomp [2], and Linux security modules [3], to bolster the security of containers and Kubernetes [4] clusters. Nevertheless, these solutions were not explicitly tailored for container scenarios, and their inherent limitations imposed restrictions on both the effectiveness of security measures and their practical application in container and Kubernetes systems.

A recent work [5] has utilized eBPF technology in container security to enhance it by leveraging the capabilities of eBPF in the kernel. Existing eBPF-based access control techniques largely concentrate on monitoring system calls and specific kernel functions, principally aimed at mitigating the impact of post-exploitation mitigation. We contend that access control methods based on Linux security modules are crucial for enhancing container runtime security because they reduce the attack surface. Furthermore, when designing a security protection system using eBPF, it is essential to predefine all the necessary hook points and create the corresponding callback functions. This enables ordinary security technicians to easily define security behaviors using the rule language. By parsing the rule files of security researchers, the system can effectively achieve security protection. Hence, in the event of a more detailed alteration in security requirements, such as the emergence of a recent CVE, it becomes impracticable to promptly implement the policy modification for real-time blocking.

Hence, to improve the comprehensiveness and flexibility of eBPF in container mandatory access control, this paper proposes BPFGuard, an eBPF-based container security mandatory access control protection system.. BPFGuard classifies container monitoring and mandatory access control into three categories: system calls, LSM hooks, and kernel functions. The approach utilizes eBPF to construct a container runtime access control mechanism with multiple levels of granularity. It is seamlessly integrated with Kubernetes to retrieve pertinent metadata. BPFGuard enables users to dynamically intercept kernel functions during runtime and perform associated actions, such as privilege escalation for capabilities and namespaces, by utilizing a unified

Received 22 August 2024; revised 25 October 2024; accepted 13 November 2024. Date of publication 24 March 2025; date of current version 9 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 61972108, Grant 62272121, and Grant U20B2046 and in part by the National Key Research and Development Plan under Grant 2023YFB3107304, and in part the Major Key Project of PCL under Grant PCL2024A05. Recommended for acceptance by Y. Wang. *(Corresponding author: Hui Lu.)*

Hui Lu, Dawei Hu, Shen Su, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 511370, China (e-mail: luhui@gzhu.edu.cn; hudawei@e.gzhu.edu.cn; sushen@gzhu.edu.cn; tianzhihong@gzhu.edu.cn).

Xiaojiang Du is with the Department of Electrical and Computer Engineering, Stevens Institute of Technology, Hoboken, NJ 07030 USA (e-mail: dxj@ieee.org).

Digital Object Identifier 10.1109/TCC.2025.3551838

2168-7161 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

routing strategy. This allows us to quickly prevent harmful attacks at the most appropriate level in the kernel when new vulnerabilities are identified. Additionally, based on the protection of the operating system, we categorize the behavior of kernel function hook functions into process characteristic matching, hook function parameter filtering, and action execution.

In summary, we make the following contributions:

- We propose a container mandatory access control mechanism with three granularity levels: system calls, LSM hooks, and kernel functions. We implemented the prototype BPFGuard based on eBPF.
- We propose a unified routing and function tail call chain utilizing eBPF, allowing users to dynamically hook any kernel function while the system is running and perform relevant actions.
- We have successfully integrated BPFGuard with Kubernetes, allowing for the deployment of BPFGuard on Kubernetes and the retrieval of pertinent metadata from the Kubernetes environment.
- We validated the security and effectiveness of BPFGuard using CVE-2022-0492 and demonstrated its usability through performance benchmarking and comparative experiments with other eBPF-based security protection systems.

II. Background

A. Namespace and Container

Namespaces [6] are a method employed to segregate and oversee system resources. Within the operating system, it is imperative to provide isolation between various processes and resources to mitigate the risk of interference and conflicts. Namespaces offer a means of segregating global resources into distinct and autonomous domains, enabling resources within each namespace to live autonomously without being affected by other namespaces. For instance, inside the Linux operating system, namespaces can be employed to segregate processes, networks, file systems, users, and other resources. This allows for improved resource allocation and heightened security measures.

Containers are a type of virtualization technique that produces lightweight virtual environments at the operating system level. They package applications and their runtime dependencies together. Containers provide substantial benefits in terms of segregating and packaging applications. Docker is a widely used container technology that utilizes Linux features like namespaces and control groups [6] to provide separate runtime environments. An application can operate within a container, which provides a distinct and isolated environment that is independent from the host system. Consequently, applications within a container are able to possess their own distinct file system, network stack, process space, and other resources, which are segregated from both other containers and the host system.

Container technology offers numerous benefits. First and foremost, containers ensure consistent functionality across many contexts, effectively resolving the common problem of 'it works on my machine.' Furthermore, containers facilitate swift deployment and scaling, empowering developers to expedite application delivery. In addition, containers enhance resource consumption by leveraging the host operating system's kernel, hence removing resource inefficiencies observed in virtual machines.

By utilizing namespaces and containers, developers may streamline the process of constructing, evaluating, and launching applications. Namespaces enhance the level of isolation in containers, hence enhancing the security and stability of programs during runtime. In addition, container technology facilitates the utilization of container orchestration technologies such as Kubernetes, which may automate the administration of extensive container clusters, enabling optimal availability and flexible scalability.

B. Linux Security Module

Linux Security Modules (LSM) is a security framework implemented at the kernel level, which allows for the expansion and customization of security rules within the Linux operating system. System administrators or developers can utilize various security measures based on individual requirements, hence improving system security. The fundamental concept underlying LSM is to decouple security determinations from the kernel and transform them into modular components that may be customized to suit specific security requirements.

The main objective of LSM is to enhance the adaptability of the Linux kernel to cater to different application scenarios and security requirements. System administrators have the ability to establish customized security policies for various system resources, such as files and processes, by incorporating and activating different LSM modules. Every LSM module has the capability to deliver security determinations, access control, auditing, and various other functions.

The fundamental principles of LSM encompass:

- *Hook mechanism*: The core principle of LSM is to insert hooks at critical points, which are called when system operations occur. These operations include process creation, system call execution, file access, and more. Each hook is a trigger point for specific events, and LSM can register and implement these hooks to perform security policy checks.
- *Hook invocation*: When a system operation triggers a hook, LSM calls the appropriate hook processing function based on the loaded LSM module. These functions can perform custom security checks, such as access control and permission verification. Based on the result of the hook call, LSM can allow, deny, or modify the system operation.
- *Security decisions*: Hook processing functions make security decisions, determining whether a process is allowed to perform a specific operation. These decisions may be based on predefined security policies, rules, and configurations. For example, when a process requests file access, LSM can check process labels, file labels, and rules to decide whether access is allowed.
- *Security policies*: Each LSM module can implement different security policies. For example, SELinux [7] uses Mandatory Access Control (MAC) to define labels and policies, while AppArmor uses process profiles to restrict resource access. Developers can choose the appropriate LSM module and configure policies based on their requirements.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

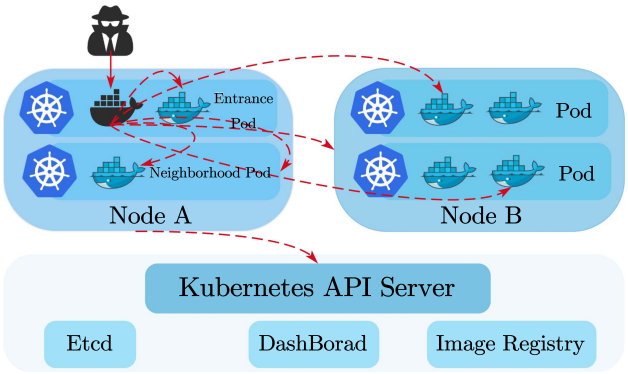


Fig. 1. Intruders perform lateral movement within the Kubernetes cluster through the compromised container.

LSM operates by registering bespoke processing functions on crucial system operation hooks, enabling intervention and customization of system behavior. The Linux kernel is capable of implementing several security policies in order to improve system security and isolation. Nevertheless, the very adaptable characteristics of LSM necessitate a profound understanding and expertise for its configuration and management.

C. eBPF

eBPF, short for extended Berkeley Packet Filter, is a virtual machine that operates within the Linux kernel. It expands and improves the functionality of the kernel without making changes to the kernel source code. This is achieved using a hook system and the loading of dynamic code.

The fundamental tenet of eBPF is to place hooks at crucial junctures within the kernel. These hooks activate relevant eBPF applications upon the occurrence of particular circumstances. For instance, an eBPF program can be activated upon the arrival of network packets or the execution of system calls. eBPF can manipulate and regulate kernel behavior by utilizing these hooks, hence facilitating the implementation of versatile functional enhancements.

In order to guarantee the stability and security of the system, eBPF programs undergo a process of security verification prior to being loaded and executed. This validation serves as a safeguard against the infiltration of harmful code that could compromise the kernel. After being validated, eBPF applications are performed by the BPF virtual machine. The primary function of this virtual machine is to read and execute eBPF program instructions, effectively managing performance demands while maintaining a high level of security.

In order to mitigate the risk of eBPF programs excessively utilizing system resources, they are subjected to restrictions regarding both computational time and memory use. These constraints guarantee the stability and efficiency of the system.

eBPF facilitates the exchange of data and memory between the kernel and user space. eBPF programs can access kernel information and save data in the kernel for examination by user space tools.

While initially intended for network packet filtering [8], eBPF has since been adopted in many fields. It has multiple applications like as performance analysis, debugging, container monitoring, security upgrades, and more.

eBPF principles facilitate the execution of customized code within the kernel, hence permitting the dynamic expansion of kernel functionality. Linux systems benefit from increased flexibility and scalability through the utilization of this utility, which serves as a robust solution for system monitoring, debugging, performance analysis, and security enhancement.

III. Movtivation

A. Container Risk

Containers are exposed to higher hazards compared to traditional virtual machine and physical machine settings, mostly due to their distinct architecture and characteristics. Containers utilize Namespaces and cgroups to attain isolation, thereby fundamentally sharing the kernel. When a container shares the host's kernel, file system, and network resources, it expands the potential for attacks. If an attacker successfully compromises a container, the processes operating on the host may also become vulnerable. Furthermore, in the container build phase, the inclusion of various components and packages increases the likelihood of unchecked packages containing vulnerabilities, or the possibility of malicious code being added during the build process. Containers can be vulnerable to attacks during runtime. Moreover, inadequate permission settings or network setups while running containers can potentially expose confidential information, hence intensifying security vulnerabilities [9], [10], [11], [12], [13], [14].

Pods have a vital function as the fundamental scheduling unit in Kubernetes, a robust container orchestration framework. A pod oversees numerous containers that operate collaboratively on a single node, utilizing the finite resources allocated by the host operating system. This sharing method enhances resource efficiency and fosters interconnection and cooperation among containers. However, this tight connection also presents security obstacles, as the interactions among containers have the ability to propagate flaws. Container escape concerns can pose a significant threat to the entire Kubernetes cluster in severe instances.

Ensuring the security of containers is crucial in a Kubernetes context. Fig. 1 demonstrates that when a container in a Kubernetes cluster is susceptible to an escape vulnerability, it poses a direct risk to the stability and data integrity of that container. Moreover, it can utilize the targeted container as a central point to infiltrate and compromise additional containers inside the cluster. This cascading effect can exert a substantial influence on the overall stability of the system, resulting in problems such as the unauthorized disclosure of sensitive data and system failures. To ensure the integrity, stability, and safety of user data in a Kubernetes cluster, it is necessary to carefully configure, monitor, and promptly patch vulnerabilities in container security. These measures are essential given the current circumstances [15], [16], [17], [18], [19].

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

B. Problems With Linux Kernel Security Methods

The existing predominant methods for enforcing container security can be primarily classified as ways based on Capabilities and Seccomp, approaches based on Linux security modules, and approaches based on eBPF.

First, both Capabilities and Seccomp have comparable problems. Configuring access control granularity for Capabilities and Seccomp necessitates substantial expertise. Otherwise, there is a potential for either too permissive configurations, which can result in serious vulnerabilities, or extremely restrictive configurations that may cause application failures. Furthermore, in real-world Kubernetes situations involving a substantial quantity of containers, the manual handling and setup of Capabilities and Seccomp become unfeasible. Furthermore, normal containers within the cluster are incapable of protecting against assaults from containers that have escaped.

Furthermore, Linux security modules, including the well-known SELinux, have faced criticism due to their intricate nature and challenging configuration process, which has resulted in reduced usability. Moreover, these approaches are restricted by the capabilities of Linux security modules, which have limited awareness and enforcement support for containers and Kubernetes scenarios. Additionally, without proper safeguards, the use of these methods may impact kernel stability in a cluster environment.

Methods utilizing eBPF provide superior hooking capabilities and programmability in comparison to Linux security components. Although eBPF has several benefits, it still has limitations, including difficulties in creating updated policies for emerging vulnerabilities or meeting more specific security requirements. Furthermore, merely limiting system calls is inadequate in addressing kernel vulnerabilities as it fails to accurately detect the real-world occurrence of the vulnerability, resulting in substantial consequences for the system. Hence, it is imperative to have a more profound level of detection for kernel functions.

Overall, the prevailing approaches to enforcing container security face their own distinct obstacles. Configuring Capabilities and Seccomp in big Kubernetes clusters can be challenging and impractical. Linux security components, such as SELinux, encounter challenges related to intricacy and insufficient backing for container contexts, which could potentially affect the stability of the kernel. Although eBPF possesses sophisticated functionalities, it nevertheless faces constraints when it comes to dynamically customizing rules for emerging vulnerabilities and fulfilling more detailed security needs. Enhanced detection systems for kernel functions are crucial, particularly in the context of kernel vulnerabilities.

C. Goals

The objective of this work is to investigate a comprehensive security protection system for container runtimes, utilizing eBPF technology, that can be effortlessly incorporated into Kubernetes. The objective is to tackle the constraints of conventional Linux security mechanisms, like Capabilities, Seccomp, and Linux security modules, by utilizing the benefits of eBPF to overcome their shortcomings as well as the restrictions of eBPF itself. Within this framework, our main focus is on the system's security capabilities and the assessment of its performance.

The system's security capabilities must possess, at the very least, the following attributes:

- Utilize a classification-trigger method to implement security regulations for system calls, LSM hook functions, and kernel functions, facilitating synchronized responses to events, rather than relying on a restricted number of enforcement points only for system calls.
- Attain comprehensive integration with Kubernetes, guaranteeing robust security measures at the cluster level.

Regarding performance, the system must satisfy the following criteria:

- Ensure fast action is taken in response to system states to effectively address emerging threats. Excessive latency can have a negative effect on the responsiveness of real-time systems. Therefore, it is important to ensure that the response time stays within an acceptable range.
- Efficiently manage anticipated workloads within the system without compromising the overall performance of the system. The system must ensure a consistent and controlled rate of resource consumption as the cluster expands, preventing excessive growth in resource usage that could negatively affect the delivery of regular business services.

To summarize, the goal is to provide a container security solution that achieves a harmonious equilibrium between delivering strong security functionalities and ensuring optimal performance. The system must be capable of accommodating the intricate characteristics of containerized environments and effectively operate even when faced with heavy workloads, guaranteeing the security and accessibility of containers and the entire Kubernetes cluster.

IV. Solution

A. BPFGuard Implementation on Cloud Native

This paper suggests a solution to address the issues of ineffective integration and the inability to access crucial Pod-related information and achieve real-time blocking in Kubernetes environments. The proposed solution involves a deep integration approach using eBPF, implemented through DaemonSet and an observer pattern, as an alternative to traditional LSM-based security measures like SELinux and Smack [20]. BPFGuard achieves this integration by utilizing user-configured files to enable real-time blocking. A Kubernetes DaemonSet ensures that a specific Pod runs on every node, commonly used for tasks like log collection and monitoring. It automatically adjusts to node changes, ensuring consistent deployment across all nodes.

BPFGuard features an Agent side that acts as a centralized security control center. Users transmit policy files to the Agent side. In order to resolve the aforementioned concerns, the policy file configuration offers the capability to intercept system calls, LSM Hook functions, and all kernel operations. It also allows for argument filtering and process filtering based on various criteria such as Namespace, PID, and Capabilities. In addition, configurations are set up to implement blocking on containers that match.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

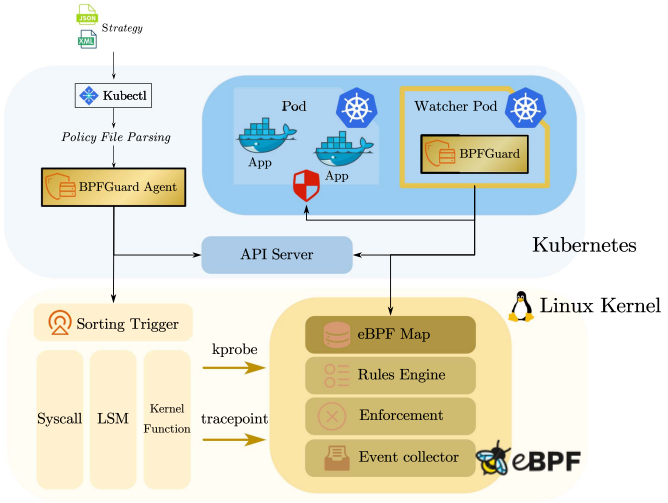


Fig. 2. BPFGuard Overall Architecture.

BPFGuard achieves deep integration with Kubernetes mostly by utilizing the DaemonSet and the observer pattern. This article utilizes BPFGuard's Kubernetes Operator to oversee the comprehensive status of BPFGuard. Each node in the cluster runs a BPFGuard pod, which is deployed via a DaemonSet. This pod not only runs the eBPF programs discussed earlier, but also includes a Watcher component. The Watcher obtains relevant information by initially retrieving a complete list of resource objects from the Kubernetes API Server and storing it locally in a cache using the SharedInformer. The SharedInformer is a high-efficiency mechanism that not only caches the initial data but also continuously watches for changes, automatically syncing the cache with the API Server. It efficiently tracks updates, additions, or deletions of resource objects without requiring constant API polling, reducing overhead. The BPFGuard Agent leverages the Watcher, accessing this cache to retrieve up-to-date Kubernetes-specific data, such as pod information, whenever needed.

The classification-trigger pattern is a vital element in BPFGuard. Policy execution is divided into three categories: system calls, LSM Hooks, and kernel functions. Each category has matching triggers, which mostly consist of eBPF programs and eBPF Maps, eBPF Map is an efficient key-value data structure used in the kernel for storing and sharing data, facilitating communication between user space and kernel space. It comes in various types, including hash tables, arrays, and queues, allowing eBPF programs to dynamically access and modify data at runtime.Once the Agent side finishes parsing the policy file, it generates a corresponding trigger object. The algorithm follows the same steps as described in Algorithm 1.

The paper proposes a way for binding and executing eBPF programs with kernel function triggers. This method primarily relies on the unified routing and function tail call chain approach, which is explained in the subsequent sections. The kernel's trigger categories are essentially composed of three eBPF-based components: the event collector, rule engine, and security executor. The event collector's primary function is to gather events within the kernel and transmit them to the application layer via

Algorithm 1: Policy Enforcement.

procedure ENFORCEPOLICY(user_pol_file)
  for all $\text{pol}_i \in$ user_pol_file **do**
    $CInfo_i \leftarrow$ GetContainerInfo(pol_i.container)
    $PSet_i \leftarrow \{\}$
    for all $t_j \in \text{pol}_i$.types **do**
      if $t_j ==$ syscall **then**
        $Cfg_j \leftarrow$ GetSyscallCfg(pol_i)
        $PSet_i \leftarrow PSet_i \cup \{\text{SysAct}(Cfg_j) \cup \text{Cfg}_j\}$
      else if $t_j ==$ LSM_hook **then**
        $Cfg_j \leftarrow$ GetLSMCfg(pol_i)
        $PSet_i \leftarrow PSet_i \cup \{\text{LSMAct}(Cfg_j) \cup \text{Cfg}_j\}$
      else if $t_j ==$ kernel_func **then**
        $Cfg_j \leftarrow$ GetKernelFuncCfg(pol_i)
        $PSet_i \leftarrow PSet_i \cup \{\text{KernelAct}(Cfg_j) \cup \text{Cfg}_j\}$
      end if
    end for
    $EData_i \leftarrow \{\text{container}_i : CInfo_i, \text{policys}_i : PSet_i\}$
    SendDataToMap($EData_i$)
    if $t_j ==$ syscall **or** $t_j ==$ LSM_hook **then**
      EnforcePolicy($CInfo_i, EData_i$)
      return
    end if
    if pol_i.hook_type == kprobe **then**
      $\text{Hooks}_i \leftarrow$ GenerateHooks($CInfo_i, \text{pol}_i$.func)
      AttachKprobes($CInfo_i, \text{Hooks}_i$)
    else if pol_i.hook_type == tracepoint **then**
      $\text{Hooks}_i \leftarrow$ GenerateHooks($CInfo_i, \text{pol}_i$.func)
      AttachTracepoints($CInfo_i, \text{Hooks}_i$)
    end if
    generateTrigger($CInfo_i, EData_i$)
  end for
end procedure

eBPF Maps. The existing pod reacts to queries originating from the Agent side. The rule engine and security executor have the responsibility of executing security policies deployed by the user in order to mitigate attacks.

The kernel's event collector transmits data regarding activated functions, such as function names and arguments, to eBPF Maps. The Watcher pod consolidates essential information at the application layer and outputs it to log files.

The overall architecture, which encompasses the foundational structure and the interplay of various system components, is meticulously illustrated in Fig. 2

B. eBPF-Based Mandatory Access Control

LSM hooks in the Linux kernel are employed to limit access to internal kernel entities such as inode, tasks, files, devices, and IPC. LSM hook-based approaches necessitate administrators to determine whether subject S has the capability to execute kernel operation OP on internal kernel object OBJ. By utilizing LSM hooks and setting suitable policies, these solutions effectively decrease the vulnerability of the system to attacks. BPF-LSM

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: File Open LSM Hook.

```
1: function GuardFileOpenfile
2:   task ← bpf_get_current_task()
3:   mntNamespace ← getTaskMntNamespaceID(task)
4:   policyMap ← getFilePolicyMap(mntNamespace)
5:   filePath ← BPF_CORE_READ(file, file_path)
6:   parseFilePath(filePath.dentry, filePath.mnt)
7:   perms ← getFilePerms(file)
8:   returnMatchPolicyForFile(policyMap, perms)
9: end Function
10: function MatchPolicyForFile(policyMap, filePerms)
11:   for index ← 0 to FILE_POLICY_MAP_MAX do
12:     rule ← getFileRule(policyMap, index)
13:     if rule.permissions&amp;amp; filePerms then
14:       if pathMatchesRule(rule) then
15:         return−EPERM
16:       end if
17:     end if
18:   end for
19:   return0
20: end Function
```

not only possesses this benefit but also alleviates the limitations of the aforementioned LSM-based techniques.

Our approach was influenced by the conventional AppArmor way and we developed obligatory access control for containers using BPF-LSM, which represents the implementation of specific trigger types discussed previously. Algorithm 2 displays the pseudo-code for the hook against the file_open function.

Regarding system calls, they essentially serve as interactive bridges between the application program and the operating system kernel. However, some perilous system calls exist within the system, such as ”chmod” and ”chown”, which involve the modification of permissions, or ”ptrace”, which allows a process to spy on and control other processes. These can potentially be exploited by malicious code, posing latent security threats to the system. Hence, ensuring that only authorized entities execute dangerous privileged operations can effectively mitigate potential attack surfaces.

Nonetheless, the currently adopted enforced access control scheme based on system calls also has some significant shortcomings. Current methods often rely on the introduction of seccomp, where treatment procedures and policy impacts may vary significantly, and lack a unified implementation in kernel space, leading to a substantial increase in system complexity. This also escalates the difficulty of maintenance and updates, thereby reducing the maintainability of the system.

Consequently, the method in this section utilises the kernel's more fundamental tracking mechanism, Raw Tracepoint, to track system calls, and likewise inaugurates eBPF mounted to sys_enter.

Specifically, when an application program initiates a particular system call, it first triggers the pre-set sys_enter Raw Tracepoint in the kernel, which then calls the mounted eBPF program. Within the eBPF program, the program first checks the user's set policies through the eBPF Map, that is, the disabled system calls. Subsequently, the program compares these policies with the second parameter of the obtained struct bpf_raw_tracepoint_args parameters, namely, the system call number. Ultimately, if the comparison result coincides, the program will send a KILL signal to terminate the current process via bpf_send_signal.

C. *Unified Routing and Function Tail Call Chain*

Caused by the extensive quantity of kernel functions, kernel function triggers are distinct from those of the other two categories. Our proposed method utilizes unified routing and function tail call chains to overcome the constraints of utilizing eBPF for implementing access control. Initially, policy files are disseminated to user space via Kubernetes, network, or local direct uploads. Subsequently, the policy files are processed by the policy analysis module, wherein they undergo syntax analysis and are differentiated based on several eBPF hook techniques. Afterwards, despite the variations in each mounting point, all mounting points requested by users are connected to a shared virtual mounting point using the standardized routing mechanism. More precisely, we analyze corresponding criteria such as process name, namespace, path, five-tuple, etc., as well as filtering logic, and then record them in the eBPF BTF object. Prior to deploying the eBPF application, it is essential to preconfigure (precompile) the required eBPF bytecode file information for the respective kprobe or tracepoint. While mounting, the Linux kernel may identify the shared mounting location, enabling users to add mounting points and enforce relevant restrictions in real-time within the obligatory access control system.

According to the provided description, all policy files that are analyzed in BPFGuard and the eBPF programs generated in the sensors are attached to bpfguard_generic_kprobe.

```
SEC("kprobe/bpfguard_kernel_kprobe")
int bpfguard_kernel_hook(void *ctx)
{
    return bpfguard_kernel_filter(ctx);
}
```

Listing 1: Virtual Common Mounting Point.

Nevertheless, any strategy must tackle two crucial concerns. An issue arises while implementing the unified routing approach as it results in the loss of the original hook point function parameters at the virtual common mounting point. An additional issue arises from the requirement of precompiled eBPF programs, which renders it unfeasible to execute customized actions for the existing hook function. In order to tackle these problems, two approaches are utilized.

In order to resolve the issue of losing the original hook point function arguments at the virtual common mounting point, we suggest implementing a communication mechanism between user space and kernel space using eBPF Maps. Upon parsing the policy file within the policy module, a bespoke protocol data packet is produced. This packet contains user-specified policy information concerning modifications in Capabilities and

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MEANING OF EACH FIELD IN THE PROTOCOL

Field	Meaning
magic_number	Used to indicate a special identifier for the protocol
version	Protocol version
total_length	Length of the data packet
selector_type	The type of filter, which can be for matching Namespace, Capabilities, or for parameter filtering
type	If selector_type is Namespace or Capabilities, this field contains specific information, such as CAP_SYS_ADMIN. If selector_type is for parameter filtering, this field contains type information, such as int
op	The matching operation for the filter value, which can be equal, greater than, less than, prefix, suffix, etc.
val_length	Length of the value to be filtered
val	The value to be filtered

Namespaces permissions, limitations to be enforced when monitoring specific parameters, and policies for alert operations. Importantly, the data packet contains both user policy information and the type information of the current hook function parameters. To do this, it is necessary to create filter_map and tail_call eBPF Maps. These maps will be used to load the current protocol data packet into the eBPF Map in the kernel. Hence, users must predefine the type information of the parameters for the desired hook function. The core premise here is that all composite data structures consist of fundamental data types, enabling the protocol analysis module to extract all type information through restricted parsing. Furthermore, while performing restriction operations, we often concentrate on information that is expressed in fundamental types. However, in actual scenarios, there is no requirement to confine ourselves to these types, and we can instead choose to support only a select few crucial types.

The last phase of policy file processing entails the interaction between user space and kernel space configuration data via eBPF Maps. This encompasses a range of filtering criteria and subsequent actions that are specified in the configuration file. The primary dependencies are the filter_map and tail_call_map, as discussed above. The tail_call_map has a rule engine that consists of a collection of functions utilized by the eBPF hook to handle different parameters and attributes via Tail Call invocation. The filter_map comprises feature matching rules that align with the configurations in the policy file. These rules define specific criteria, such as the PID of processes to be limited or the parameters of eBPF program hook functions, which are known as filter rules. Here is an uncomplicated illustration:

```
filter = [magic_number]
         [version][total_length]
         [selector_type]
         [type|op|val_length|val][action]
         [selector_type][index]
         [arg_length][arg][action]
```

Listing 2: Protocol Example.

The meaning of each of these fields is shown in Table I.

In order to resolve the problem of the Hook functions' lack of specialization, we utilize the previously mentioned custom protocol to determine argument types and utilize the function tail call chain technique to circumvent the constraints of eBPF programs. In order to accomplish this, we have developed an operational framework for limiting container operations, taking into account the existing state of common vulnerabilities and exposures (CVEs) related to containers. This model includes a range of restriction operations, such as matching process IDs, modifying Capabilities and Namespaces permissions, and filtering parameters.

Additionally, it is important to mention that eBPF prohibits the calling of non-eBPF programs in the kernel space and the running of loops with an uncertain number of iterations. Hence, we employ the eBPF tail call method to execute function calls. When a user configures the policy file to block three features and filter two arguments, we execute five tail call operations.

D. eBPF-Based Security Enforcer

If the primary objective of the policy file for BPFGuard is to impose required access control, the system must furthermore have a security enforcer that is based on eBPF. If a filter fails to pass after going through the rule engine and event collector, the procedure proceeds to the security enforcer. The security enforcer then executes the obligatory actions outlined in the policy file. BPFGuard principally supports two categories of mandatory actions: transmission of signals and overriding of return values.

Signal sending involves using the eBPF helper function [21] bpf_send_signal to send a specified signal within the kernel to the current process. The most straightforward operation is to send a Kill signal to terminate the current process that has been matched by the rule engine. For example, if any process with a PID not equal to 1 or 0 attempts to read or write to the /etc/passwd file, it can be considered a malicious process, and a Kill signal can be sent to terminate it.

Return value overriding relies on the bpf_override_return implemented through kernel fault injection [22], allowing for the modification of a function's return value. While sending a

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

Kill signal to terminate the current process is a highly effective and straightforward operation, there are cases where it might be preferable to return a failure-indicating return value. This, in turn, leaves the decision of termination or continued execution to the current process's program. For instance, when a process with a PID not equal to 1 or 0 uses the read system call to access the /etc/passwd file, it can return -1 or 0.

V. Evaluation

Within this part, we have verified the efficacy of BPFGuard by employing CVE-2022-0492. We have additionally furnished a concise overview of BPFGuard's load performance testing and juxtaposed its performance with that of Tracee [23], a widely-used container runtime security protection technology.

A. Security Effectiveness

In 2022, Linux maintainers revealed a security flaw in the Linux kernel that allows for privilege escalation. The vulnerability, designated with the common vulnerabilities and exposures ID CVE-2022-0492 [24], has been classified as a high severity issue with a rating of 7.8.

The vulnerability lies within the cgroup_release_agent_write function in the Linux kernel's cgroup-v1.c file. This vulnerability, under specific circumstances, enables privilege escalation by exploiting the cgroups v1 release_agent capability, resulting in the unintended bypassing of namespace isolation.

The release_agent file is a notable feature of cgroups v1. This file can be utilized by administrators to set up a release agent software that executes when processes within a cgroup conclude. To accomplish this, one must write the desired release agent path to the release_agent file, as demonstrated in the example below:

```
$ echo /bin/my-release-agent \
  >/sys/fs/cgroup/memory/release_agent
```

It's important to note that the release_agent file is only visible in the root cgroup directory and only affects its sub-cgroups. By writing to the notify_on_release file, each sub-group can be configured to either trigger or not trigger the release agent when a process terminates within it. The following command enables the notify_on_release functionality for the a_child_cgroup cgroup:

```
$ echo 1 > /sys/fs/cgroup/cgroup-v1 \
  /a_child_cgroup/notify_on_release
```

This command assigns the value of 1 to the notify_on_release parameter, which signifies that the release agent will be activated when a process stops within the specified a_child_cgroup cgroup.

Upon the termination of a process, the kernel verifies if its cgroups have the "notify_on_release" feature enabled. When this functionality is activated, the designated "release_agent" executable file is run. Currently, the release agent operates with the utmost privileges, essentially functioning as a root process

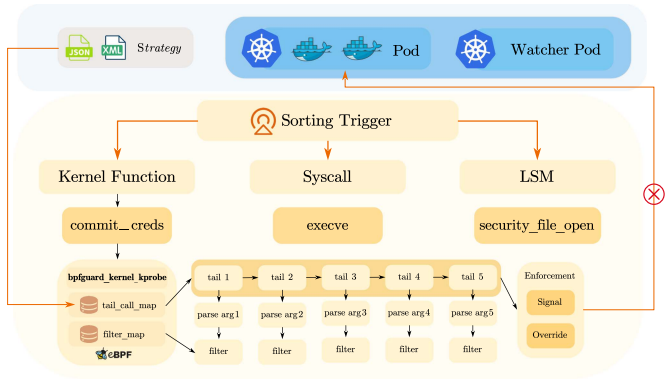


Fig. 3. Implementation of mandatory access control for classification triggers based on eBPF.

```
root@cve-2022-0492:/# bash ./exp.sh "touch /escape_success"
[-] You donot have CAP_SYS_ADMIN, will try
[+] Escape Success with unshare!
```

Fig. 4. Escape failed.

with complete permissions within the first namespace. Hence, configuring the release agent is regarded as a privileged activity as it grants users the ability to designate the specific binary file that will execute with full root rights.

The precise procedure is depicted in Fig. 4.

To get detailed implementations, please consult the publicly accessible EXP code [25].

The fundamental reason for the CVE-2022-0492 issue is the absence of adequate validation. Linux does not verify whether the process that sets the "release_agent" file possesses the required administrative privileges, especially the CAP_SYS_ADMIN permission. This vulnerability is exploited by exploiting the "mount cgroup" operation in order to execute malicious activities on the cgroup "release_agent" and escape from its restrictions.

To tackle this problem, a direct method is to employ BPFGuard to either limit the functionality of the "__x64_sys_mount" kernel function or supervise alterations in capabilities. This will effectively hinder or identify potential attempts to exploit vulnerabilities.

```
apiVersion: crd.bpfguard.com/v1beta1
kind: BPFGuardPolicy
metadata:
  name: "sys-mount-kill"
spec:
  # int mount(...);
  - type: kernel-function
      call: "__x64_sys_mount"
    selectors:
      - action: Kill
```

Listing 3: Policy Example.

From Fig. 4, you can observe that the exploit fails to achieve escape upon running it again.

Indeed, BPFGuard endows the system with the capability of fine-grained mandatory access control. Consequently, system

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SERVER CONFIGURATION INFORMATION

Kubernetes Version	1.24
CPU	Intel(R) Core(TM) i5-8250U 1.60GHz
Memory Size	8GB
Operating System	Ubuntu 22.04

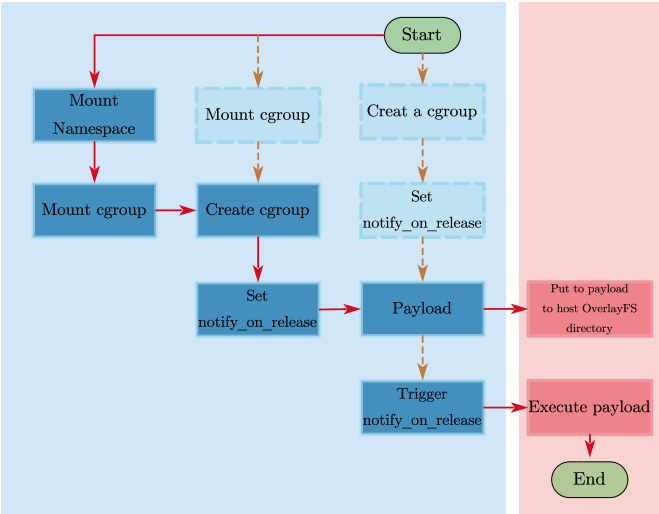


Fig. 5. CVE-2022-0492 Vulnerability Exploitation Flow.

administrators can scrutinize additional characteristic behaviors associated with CVE-2022-0492, and subsequently employ BPFGuard to impede the ongoing process. This blocking methodology remains consistent when addressing other vulnerabilities.

B. Performance

In order to evaluate the performance data of BPFGuard, this study chose five widely used applications, such as databases, servers, and cache databases, for conducting load performance tests. The comparison between BPFGuard and Tracee, which are both container runtime security protection systems, was conducted using Nginx. The server setups and Kubernetes versions utilized in the trials are enumerated in Table II.

We implemented BPFGuard on Kubernetes as a DaemonSet, retrieved container images of the desired programs for monitoring, and executed them. Currently, the Watcher pods on the nodes have the task of monitoring the specified system calls, LSM Hooks, or kernel functions mentioned in the policy files.

We conducted an experiment where we observed the performance of Redis, MySQL, Nginx, httpd, and Memcached. We used appropriate stress testing tools to deliberately increase the workload on BPFGuard and assessed the apps accordingly. The functions under surveillance are enumerated in Table III.

We utilize Kubernetes' Kubectl to examine the present CPU performance conditions of the node.

As depicted in Fig. 6, upon activating BPFGuard and configuring it with 0, 5, 10, and 20 policies respectively during

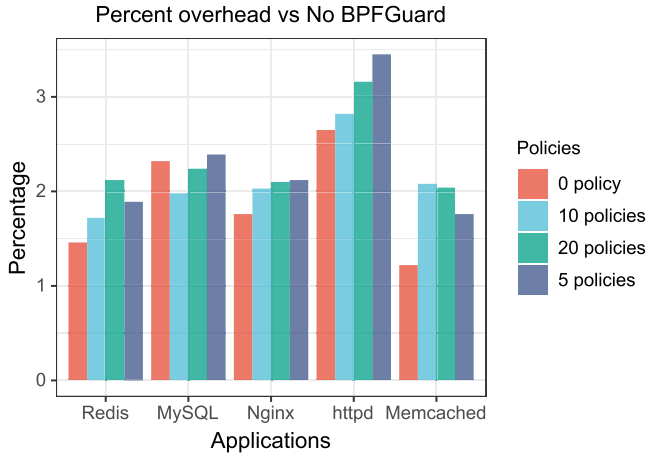


Fig. 6. Percent overhead versus No BPFGuard.

multiple stress tests performed on Redis, the system's average CPU performance loss increased by 1.46%, 1.89%, 1.72%, and 2.12% compared to when BPFGuard was not utilized. When BPFGuard was activated and configured with 0, 5, 10, and 20 policies respectively during numerous stress tests conducted on MySQL, the system's average CPU performance loss increased by 2.32%, 2.39%, 1.98%, and 2.24% compared to when BPFGuard was not adopted. Following the activation of BPFGuard and its configuration with 0, 5, 10, and 20 policies respectively during repeated stress tests on Nginx, the system's average CPU performance loss rose by 1.76%, 2.21%, 2.03%, and 2.1% compared to the average performance loss when BPFGuard was not operational. Upon the activation of BPFGuard, configured with 0, 5, 10, and 20 policies respectively during multiple stress tests on httpd, the system's average CPU performance loss remarkably escalated by 2.65%, 3.45%, 2.82%, and 3.16% compared to when BPFGuard was not deployed. Lastly, when BPFGuard was activated and configured with 0, 5, 10, and 20 policies respectively during successive stress tests performed on Memcached, the system's average CPU performance loss increased by 1.22%, 1.76%, 2.08%, and 2.04% in contrast to non-usage of BPFGuard. In summary, the minimum performance loss in the aforesaid benchmark tests is 1.22%, with the maximum being 3.45%—averaging out to a loss of 2.16%. As such, the impact on system performance is relatively minor, affirming the ability to achieve higher levels of security at a substantially minimal expense.

Furthermore, we subjected the Tomcat application as the monitored target program (with the monitored system calls or kernel functions aligning with the experiments conducted on Nginx as described earlier) to stress tests involving Tracee[14] and BPFGuard. A comparative analysis of the performance in the user space between the two was conducted. The examination also employed Kubernetes's Kubectl tool to assess the current node's CPU and memory performance.

As shown in Fig. 7, BPFGuard's stable CPU usage (in millicore) at request volumes of 1000, 2000, 5000, 10,000, 25,000, and 50,000 are 49, 87, 124, 207, 311, and 405, respectively. Tracee's stable CPU usage (in millicore) at request

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
FUNCTIONS TO BE MONITORED FOR DIFFERENT APPLICATIONS

Application	Monitored Functions
Redis	bind, listen, accept, open, read, write, malloc, pthread_create, gettimeofday, clock_gettime, ioctl
MySQL	bind, listen, accept, open, read, write, fcntl, mmap, epoll_wait, ioctl, fork
Nginx	bind, listen, accept, open, read, write, recv, send, epoll_wait, waitpid, fork, execve, lseek
httpd	bind, listen, accept, open, read, write, recv, send, epoll_wait, ioctl, fork
Memcached	bind, listen, accept, open, read, write, recv, send, epoll_wait, malloc, free

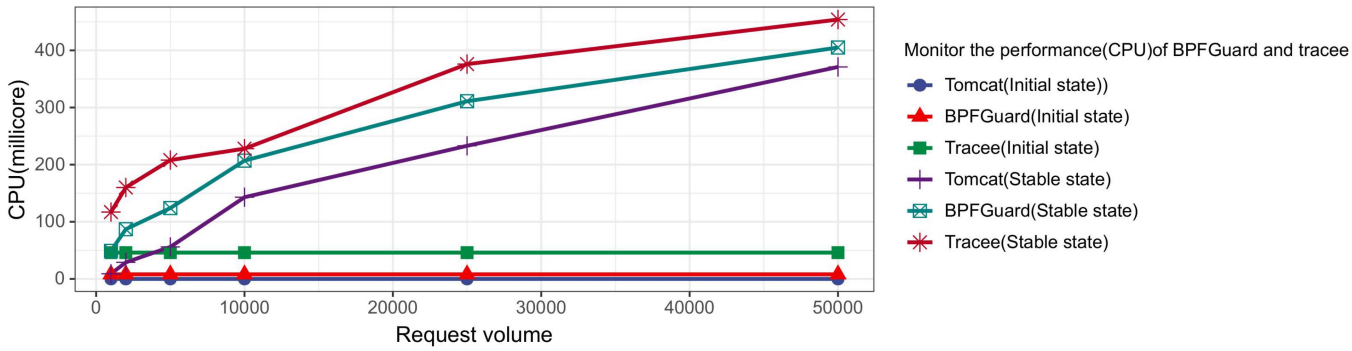


Fig. 7. Monitor the performance(CPU) of BPFGuard and tracee.

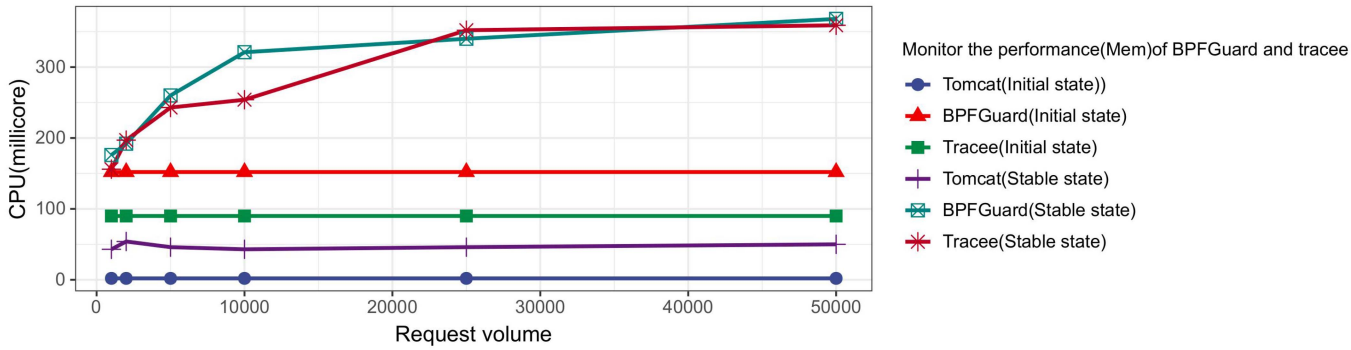


Fig. 8. Monitor the performance(Mem) of BPFGuard and tracee.

volumes of 1000, 2000, 5000, 10,000, 25,000, and 50,000 are 117, 160, 208, 228, 376, and 454, respectively.

As shown in Fig. 8, BPFGuard's stable memory usage (in Membibytes) at request volumes of 1000, 2000, 5000, 10,000, 25,000, and 50,000 are 176, 192, 260, 321, 340, and 368, respectively. Tracee's stable memory usage (in Membibytes) at request volumes of 1000, 2000, 5000, 10,000, 25,000, and 50,000 are 156, 197, 243, 254, 352, and 359, respectively.

In conclusion, when comparing BPFGuard to Tracee under varying request volumes, the CPU usage performance increased by 58%, 45%, 40%, 9%, 17%, and 10%, respectively, thus demonstrating a superior performance to Tracee. As for memory usage, due to differences in event handling methods, there is a degree of slight fluctuations with an average difference value of 21.6. Nevertheless, both BPFGuard and Tracee do not produce adverse effects on the container services. Additionally, due to the different stages of system operation, there will be variants in the initial status.

VI. RELATED WORK

In the context of utilizing Seccomp, Lei et al. [26] introduced the concept of phased execution for containers. They aimed to reduce the number of available system calls in containers by distinguishing short-term and long-term stages, thereby mitigating security risks. They achieved this by extending Seccomp to dynamically and automatically update the available system calls as an application transitions from the short-term initialization phase to the long-term running phase. Wan et al. [27] proposed a container sandbox exploration solution based on automated testing. During the testing phase, Seccomp is used to extract the system calls used by the container, and at runtime, only the

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

system calls used in the testing phase are allowed for program execution. Ghavamnia et al. [28] suggested using static code analysis to inspect containerized applications and their dependencies. They identified a superset of system calls necessary for the correct operation of containers and generated corresponding Seccomp system call policies.

In the realm of utilizing Linux Security Modules, Mattetti et al. [29] proposed a method to generate AppArmor policy files by tracking the container daemon during the container image construction process. The generated AppArmor policy files are then used to restrict the container's behavior during runtime. Loukidi et al. [30] introduced an automated system to enhance the security of Docker containers, also based on AppArmor. It mainly consists of two crucial mechanisms. First, during image creation, the system creates a set of static access rules based on image creation parameters. Second, at runtime, the initial set is enhanced to further restrict threats posed by the image. One of the main issues with Linux Security Modules is their shared nature among all containers, where a single container cannot use specific Linux Security Modules independently. Therefore, Sun et al. [31] proposed Security Namespaces, allowing containers to independently define security policies. They also introduced a routing scheme for dynamically distributing operations to containers, ensuring that security policies generated by one container do not impact the host or other containers.

Concerning the use of eBPF, there is current research combining LSM and eBPF, such as KRSI [32] and BPF-LSM [33] (Berkeley Packet Filter-Linux Security Modules). By combining LSM and eBPF, these approaches enable the attachment of eBPF programs to hundreds of LSM hook points and the execution of the logic of original security modules. This enhances LSM with dynamic policy update capabilities and significantly improves performance. Landlock LSM [34] also combines the two components. Originally, the usage of security modules like LSM required privileges. Therefore, the goal of Landlock LSM is to enable non-privileged processes to protect themselves with user-passed eBPF programs. However, Landlock LSM controls what parameters are passed to user eBPF programs based on LSM, ensuring that the security of the system is not compromised by user eBPF programs. To leverage the capabilities of eBPF and enhance support for container environments, Findlay et al. [35] utilized the programmability and high-performance advantages of eBPF to implement a runtime security detection architecture for container scenarios. This allows users to restrict certain system calls and LSM hooks during runtime. Subsequently, Findlay et al. [36] proposed combining eBPF with a simpler policy language for container restriction mechanisms, aiming to integrate with existing container management systems, providing more robust restriction capabilities and a simpler policy configuration process. The Cloud Native Computing Foundation (CNCF) projects Tracee and Kuberarmor [37] have also introduced eBPF to enhance system observability, complementing Linux security modules to achieve partial system restriction capabilities.

On another front, Young et al. [38] proposed gVisor, which considers container enforcement from a different perspective, termed process virtualization. gVisor isolates malicious code within the container from accessing the Host Kernel. It implements a "kernel process" outside the kernel, providing most Linux system calls. Through clever mechanisms, it transforms containerized process system calls into access to this "kernel process," thereby improving security and isolation. Randazzo et al. [39] introduced Kata Containers, which are more radical. Essentially, Kata Containers are virtual machines compatible with the OCI container runtime standard.

VII. Conclusion

Containerized applications and Kubernetes clusters have become widely adopted technologies in cloud computing. However, their security concerns have not been comprehensively addressed. Therefore, this paper mitigates the shortcomings in container monitoring and mandatory access control using BPFGuard and integrates it deeply with Kubernetes, offering a more robust container security framework. Additionally, due to the low performance overhead of BPFGuard and the scalable nature of Kubernetes resources, BPFGuard does not adversely impact the normal operation of container services. It provides a highly secure environment at a relatively low cost.

Furthermore, the primary performance loss in the system occurs in the reporting phase after event collection. This module can be targeted for performance optimization, further reducing the overall impact on the system.

References

[1] L. Manual, "Capabilities," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://man7.org/linux/man-pages/man7/capabilities.7.html

[2] D. Documentation, "Seccomp security profiles for docker," 2019, Accessed: Apr. 1, 2025. [Online]. Available: https://docs.docker.com/engine/security/seccomp/

[3] C. Wright, C. Cowan, S. Smalley, J. Morris, and G. Kroah-Hartman, "Linux security modules: General security support for the linux kernel," in *Proc. 11th USENIX Secur. Symp.*, 2002, pp. 17–31.

[4] M. Luksa, *Kubernetes in Action*. New York, NY, USA: Simon and Schuster, 2017.

[5] L. Deri, S. Sabella, and S. Mainardi, "Combining system visibility and security using eBPF," in *Proc. 3rd Italian Conf. Cybersecur.*, Pisa, Italy, vol. 2315, Feb. 2019, pp. 1–12..

[6] R. Rosen, "Namespaces and cgroups, the basis of linux containers," in *Proc. Tut. NetDev 1.1 Tech. Con. Linux Netw.*, Seville, Spain, Feb., 2016, Accessed: Apr. 1, 2025. [Online]. Available: https://netdevconf.org/1.1/proceedings/slides/rosen-namespaces-cgroups-lxc.pdf

[7] S. Smalley, C. Vance, and W. Salamon, "Implementing selinux as a linux security module," *NAI Labs Rep.*, vol. 1, no. 43, 2001, Art. no. 139.

[8] M. Bertrone, S. Miano, F. Risso, and M. Tumolo, "Accelerating linux security with ebpf iptables," in *Proc. ACM SIGCOMM Conf. Posters Demos*, 2018, pp. 108–110.

[9] H. Lu, C. Jin, X. Helu, C. Zhu, N. Guizani, and Z. Tian, "AutoD: Intelligent blockchain application unpacking based on JNI layer deception call," *IEEE Netw.*, vol. 35, no. 2, pp. 215–221, Mar./Apr. 2021.

[10] J. Hou, F. Liu, H. Lu, Z. Tan, X. Zhuang, and Z. Tian, "A novel flow-vector generation approach for malicious traffic detection," *J. Parallel Distrib. Comput.*, vol. 169, pp. 72–86, 2022.

[11] H. Lu, C. Jin, X. Helu, X. Du, M. Guizani, and Z. Tian, "DeepAutoD: Research on distributed machine learning oriented scalable mobile communication security unpacking system," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2052–2065, 2021.

[12] Z. Gu, W. Hu, C. Zhang, H. Lu, L. Yin, and L. Wang, "Gradient shielding: Towards understanding vulnerability of deep neural networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 921–932, Second Quarter 2021.

[13] Z. Wang, K. Liu, J. Hu, J. Ren, H. Guo, and W. Yuan, "Attrleaks on the edge: Exploiting information leakage from privacy-preserving co-inference," *Chin. J. Electron.*, vol. 32, no. 1, pp. 1–12, Jan. 2023.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.

[14] X. Pang, Z. Wang, D. Liu, J. C. Lui, Q. Wang, and J. Ren, "Towards personalized privacy-preserving truth discovery over crowdsourced data streams," *IEEE/ACM Trans. Netw.*, vol. 30, no. 1, pp. 327–340, Feb. 2022.
[15] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "Xi commandments of kubernetes security: A systematization of knowledge related to kubernetes security practices," in *Proc. IEEE Secure Develop.*, 2020, pp. 58–64.
[16] M. Pan et al., "DHPA: Dynamic human preference analytics framework: A case study on taxi drivers' learning curve analysis," *ACM Trans. Intell. Syst. Technol.*, vol. 11, no. 1, pp. 1–19, 2020.
[17] N. Hu, Z. Tian, H. Lu, X. Du, and M. Guizani, "A multiple-kernel clustering based intrusion detection scheme for 5G and IoT networks," *Int. J. Mach. Learn. Cybern.*, vol. 12, pp. 3129–3144, 2021.
[18] H. Lu et al., "Research on intelligent detection of command level stack pollution for binary program analysis," *Mobile Netw. Appl.*, vol. 26, pp. 1723–1732, 2021.
[19] Z. Tian, M. Li, M. Qiu, Y. Sun, and S. Su, "Block-def: A secure digital evidence framework using blockchain," *Inf. Sci.*, vol. 491, pp. 151–165, 2019.
[20] T. L. K. documentation, "Smack," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/admin-guide/LSM/Smack.html
[21] L. Manual, "BPF-helpers," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://www.man7.org/linux/man-pages/man7/bpf-helpers.7.html
[22] T. L. K. documentation, "Fault injection capabilities infrastructure," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://docs.kernel.org/fault-injection/fault-injection.html
[23] A. Security, "Tracee: Runtime security and forensics using EBPF," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://aquasecurity.github.io/tracee/v0.11/
[24] N. V. Database, "CVE-2022-0492 detail," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2022-0492
[25] Chenaotian, "CVE-2022-0492," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://github.com/chenaotian/CVE-2022-0492
[26] L. Lei et al., "Speaker: Split-phase execution of application containers," in *Proc. 14th Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, Bonn, Germany, Springer, 2017, pp. 230–251.
[27] Z. Wan, D. Lo, X. Xia, L. Cai, and S. Li, "Mining sandboxes for linux containers," in *Proc. IEEE Int. Conf. Softw. Testing, Verification Validation*, 2017, pp. 92–102.
[28] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 443–458.
[29] M. Mattetti, A. Shulman-Peleg, Y. Allouche, A. Corradi, S. Dolev, and L. Foschini, "Securing the infrastructure and the workloads of linux containers," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2015, pp. 559–567.
[30] F. Loukidis-Andreou, I. Giannakopoulos, K. Doka, and N. Koziris, "Docker-sec: A fully automated container security enhancement mechanism," in *Proc. IEEE 38th Int. Conf. Distrib. Comput. Syst.*, 2018, pp. 1561–1564.
[31] Y. Sun, D. Safford, M. Zohar, D. Pendarakis, Z. Gu, and T. Jaeger, "Security namespace: Making linux security frameworks available to containers," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1423–1439.
[32] K. Singh, "Mac and audit policy using eBPF," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://lwn.net/ml/linux-kernel/20191220154208.15895-1-kpsingh,chromium.org/
[33] T. L. K. documentation, "LSM BPF programs," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://docs.kernel.org/bpf/prog_lsm.html
[34] M. Salaün, "Landlock LSM: Kernel documentation," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://docs.kernel.org/security/landlock.html
[35] W. Findlay, A. Somayaji, and D. Barrera, "BPFBOX: Simple precise process confinement with eBPF," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, 2020, pp. 91–103.
[36] W. Findlay, D. Barrera, and A. Somayaji, "Bpfcontain: Fixing the soft underbelly of container security," 2021, *arXiv:2102.06972*.
[37] AccuKnox, "Runtime protection for kubernetes," 2023. Accessed: Jan. 02, 2024. [Online]. Available: https://kubearmor.io/
[38] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The true cost of containing: A {gVisor} case study," in *Proc. 11th USENIX Workshop Hot Topics Cloud Comput.*, 2019, pp. 1–6.
[39] A. Randazzo and I. Tinnirello, "Kata containers: An emerging architecture for enabling MEC services in fast and secure way," in *Proc. 6th Int. Conf. Internet Things, Syst., Manage. Secur.*, 2019, pp. 209–214.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:38:21 UTC from IEEE Xplore. Restrictions apply.